

# Chapter 2: Derivatives and Gradients

Algorithms for Optimization, 2<sup>nd</sup> ed. (2026)

Kochenderfer & Wheeler  
Slides prepared for lecture

MIT Press

2026

# Chapter Outline

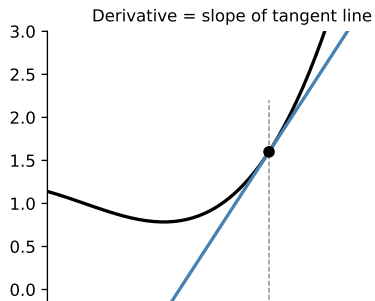
1. Scalar Derivatives
2. Derivatives in Multiple Dimensions
3. Numerical Differentiation
4. Automatic Differentiation
5. Regression Gradient
6. SPSA
7. Summary and Exercises

# Motivation: Why Do We Need Derivatives? I

Optimization is the process of finding an input value — a *design point* — that makes some objective function as small (or as large) as possible. To search efficiently, we need a way to answer the question: **“If I change my input slightly, does the function go up or down?”** That question is answered by the *derivative*.

The **derivative** of a function  $f$  at a point  $x$  tells us the *instantaneous rate of change* of  $f$  with respect to  $x$ . Concretely, it is the slope of the straight line that just touches (is tangent to) the graph of  $f$  at the point  $x$ . When the derivative is positive,  $f$  is increasing at  $x$ ; when it is negative,  $f$  is decreasing; when it is zero, we may be at a local minimum or maximum.

In one dimension the derivative is written  $f'(x)$  (read “ $f$  prime of  $x$ ”) or equivalently  $\frac{df}{dx}$  (read “ $d$ - $f$  over  $d$ - $x$ ”, Leibniz notation). In multiple dimensions — when  $f$  takes a vector of inputs — the analogous object is the **gradient**, written  $\nabla f(\mathbf{x})$  ( $\nabla$  is called *nabla* or *del*). Knowing the gradient, a gradient-based optimizer moves the design point in the direction that most rapidly decreases  $f$  — this is the foundation of virtually every modern optimizer.



## Motivation: Why Do We Need Derivatives? II

### Key Takeaway

The derivative (in 1-D) and the gradient (in higher dimensions) encode the local slope of the objective function. An optimizer uses this information to decide which direction to step next.

# Scalar Derivative: Formal Definition I

Intuitively, the derivative is “how much does  $f$  change when I nudge  $x$  by a tiny amount  $h$ ”, normalized by the size of that nudge. The formal definition takes this idea to the limit as the nudge  $h$  shrinks to zero.

## Definition of the Derivative

The **derivative**  $f'(x)$  of a function  $f : \mathbb{R} \rightarrow \mathbb{R}$  at the point  $x$  is defined as:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} = \frac{\Delta f(x)}{\Delta x}$$

# Scalar Derivative: Formal Definition II

## Notation

- $f : \mathbb{R} \rightarrow \mathbb{R}$  means  $f$  takes a single real number as input and produces a single real number as output.
- $h$  is a small real number representing the size of the perturbation (nudge) applied to  $x$ .
- $\lim_{h \rightarrow 0}$  (read “the limit as  $h$  approaches zero”) means we imagine  $h$  becoming arbitrarily small, but never actually reaching zero.
- $\Delta f(x)$  (“Delta  $f$  of  $x$ ”;  $\Delta$  is pronounced *Delta*) denotes the change in  $f$ , i.e.,  $f(x + h) - f(x)$ .
- $\Delta x$  (“Delta  $x$ ”) denotes the change in the input, i.e.,  $h$ .

## How to Read This Formula

The fraction  $\frac{f(x + h) - f(x)}{h}$  is the slope of the straight line connecting the two points  $(x, f(x))$  and  $(x + h, f(x + h))$  on the graph. As  $h$  shrinks toward zero, this secant line rotates until it touches the graph at exactly one point — the tangent line. Its slope is  $f'(x)$ .

# Scalar Derivative: Formal Definition III

Three mathematically equivalent ways to write the same limit arise from choosing which side of  $x$  to nudge toward.

## Three Equivalent Limit Forms

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad (\text{forward: nudge to the right})$$

$$= \lim_{h \rightarrow 0} \frac{f(x+h/2) - f(x-h/2)}{h} \quad (\text{central: symmetric bracket around } x)$$

$$= \lim_{h \rightarrow 0} \frac{f(x) - f(x-h)}{h} \quad (\text{backward: nudge to the left})$$

All three give exactly the same limit — they are three different ways to approach the same tangent slope. In practice, however, when  $h$  is kept *small but nonzero* (as in numerical computation), the

## Scalar Derivative: Formal Definition IV

central form turns out to be more accurate. We will see precisely why when we analyze finite differences in Section 3.

**Linear approximation interpretation.** Because the derivative is the slope of the tangent line, we can approximate  $f$  near  $x$  by that line:

$$f(x + \Delta x) \approx f(x) + f'(x) \Delta x$$

This says: if you move by a small amount  $\Delta x$  (“Delta  $x$ ”, the change in  $x$ ), the function value changes by approximately  $f'(x) \Delta x$ . The approximation is accurate when  $\Delta x$  is small relative to the scale on which the slope changes.



# Symbolic Differentiation I

The most familiar way to compute a derivative is *symbolic differentiation*: applying the rules of calculus (power rule, product rule, chain rule, etc.) to the algebraic expression for  $f$  and producing an exact algebraic expression for  $f'$ .

**Example (analogous to Example 2.1 in the textbook).** Consider the function

$$f(x) = x^2 + \frac{x}{2} - \frac{\sin(x)}{x}.$$

To differentiate this symbolically we apply:

- The **power rule**:  $\frac{d}{dx}[x^n] = nx^{n-1}$ , so  $\frac{d}{dx}[x^2] = 2x$ .
- The **constant-multiple rule**:  $\frac{d}{dx}\left[\frac{x}{2}\right] = \frac{1}{2}$ .
- The **quotient rule** on  $\frac{\sin x}{x}$ :  $\frac{d}{dx}\left[\frac{\sin x}{x}\right] = \frac{x \cos x - \sin x}{x^2} = \frac{\cos x}{x} - \frac{\sin x}{x^2}$ .

# Symbolic Differentiation II

Putting it together:

$$f'(x) = 2x + \frac{1}{2} + \frac{\sin x}{x^2} - \frac{\cos x}{x}.$$

This is an *exact* expression: there is no numerical error at all.

# Symbolic Differentiation III

## Python: symbolic differentiation with SymPy.

The Python library `sympy` can perform the same algebra automatically. Below, `diff(f, x)` applies the differentiation rules and returns the simplified expression.

### SymPy Example

```
from sympy import symbols, diff, sin
x = symbols('x')
f = x**2 + x/2 - sin(x)/x
df = diff(f, x)
# Result:
# 2*x + 1/2 + sin(x)/x**2 - cos(x)/x
```

## When symbolic differentiation shines:

Symbolic differentiation produces an exact, closed-form expression that can be evaluated at any point without any approximation error.

**Its limitation — expression swell:** For complicated nested functions the symbolic expression for  $f'$  can be *exponentially larger* than the expression for  $f$  itself (each application of the product rule doubles the number of terms). This is called *expression swell* and can make the derivative very slow to evaluate. For code-defined functions (loops, conditionals), symbolic methods do not apply at all.

## Key Takeaway

Symbolic differentiation is ideal when  $f$  has a compact algebraic formula and you need the derivative in closed form. For large or code-defined functions, automatic differentiation (Section 4) is preferred.

# The Gradient: Extending the Derivative to Many Inputs I

In optimization problems we almost always have more than one input variable. For example, we might be choosing the weights of a neural network (millions of variables) or the trajectory of a robot arm (hundreds of joint angles). The derivative must be extended to this multi-input setting.

For a function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  (“ $f$  maps an  $n$ -dimensional vector to a single real number”), we can compute the derivative with respect to each input variable separately, holding all other variables constant. Each such derivative is called a **partial derivative**.

# The Gradient: Extending the Derivative to Many Inputs II

## Definition: Gradient (eq. 2.5)

The **gradient**  $\nabla f(\mathbf{x})$  (“nabla  $f$  of  $\mathbf{x}$ ”) is the column vector whose  $i$ -th entry is the partial derivative of  $f$  with respect to the  $i$ -th input:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \in \mathbb{R}^n$$

# The Gradient: Extending the Derivative to Many Inputs III

## Notation

- $\mathbf{x} = (x_1, x_2, \dots, x_n)^\top$  (“ $\mathbf{x}$  bold”) is the  $n$ -dimensional input vector. Subscript  $i$  indexes the  $i$ -th component.
- $\frac{\partial f}{\partial x_i}$  (“partial  $f$  partial  $x$ -sub- $i$ ”) is the partial derivative: the rate of change of  $f$  when  $x_i$  changes while all other  $x_j$  (for  $j \neq i$ ) are held fixed.
- $\mathbb{R}^n$  means the set of all  $n$ -dimensional real vectors.
- The superscript  $\top$  (read “transpose”) converts a row vector to a column vector or vice versa.

# The Gradient: Extending the Derivative to Many Inputs IV

## How to Read the Gradient

The gradient  $\nabla f(\mathbf{x})$  is a vector that **points in the direction of steepest ascent** of  $f$  at the point  $\mathbf{x}$ . In other words, if you step a tiny distance  $\epsilon$  (epsilon) in the direction  $\nabla f(\mathbf{x})$ , you increase  $f$  faster than in any other direction. Conversely, stepping in the direction  $-\nabla f(\mathbf{x})$  (the *negative gradient*) decreases  $f$  most rapidly — this is the basic idea behind gradient descent.

**Worked numerical example.** Let  $f(x_1, x_2) = x_1^2 + 2x_2^2$ . The partial derivatives are:

$$\frac{\partial f}{\partial x_1} = 2x_1, \quad \frac{\partial f}{\partial x_2} = 4x_2.$$

So the gradient at the point  $\mathbf{x} = (1, 1)^\top$  is:

$$\nabla f\left(\begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 2(1) \\ 4(1) \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}.$$



# The Gradient: Extending the Derivative to Many Inputs V

This tells us: at  $(1, 1)$ ,  $f$  increases approximately 2 units per unit step in the  $x_1$ -direction, and approximately 4 units per unit step in the  $x_2$ -direction. The steepest ascent direction is  $(2, 4)^\top$ , pointing more steeply in the  $x_2$  direction because  $f$  has greater curvature there.

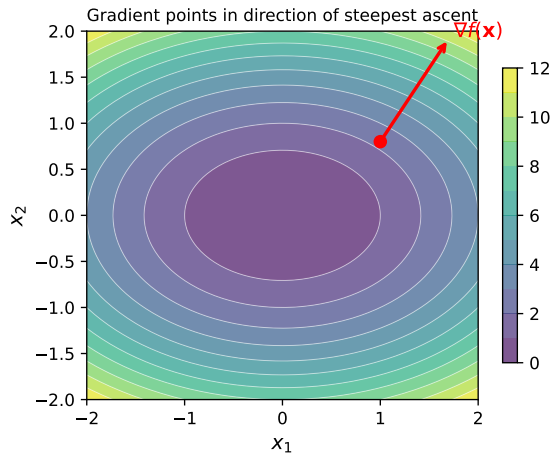
The gradient also enables a *first-order (linear) approximation* of  $f$  near  $\mathbf{x}$ :

$$f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \Delta\mathbf{x}$$

where  $\Delta\mathbf{x}$  (“Delta  $\mathbf{x}$  bold”) is a small displacement vector. The inner product  $\nabla f(\mathbf{x})^\top \Delta\mathbf{x}$  measures how much  $f$  changes when we move by  $\Delta\mathbf{x}$ .

# Gradient: Geometric Intuition I

The gradient has a beautiful geometric meaning that is easiest to see through the *level curves* (also called *contour lines*) of  $f$ . A level curve is the set of all points  $\mathbf{x}$  where  $f(\mathbf{x}) = c$  for some constant  $c$  — like the elevation contour lines on a topographic map.



## Key geometric observations:

The gradient  $\nabla f(\mathbf{x})$  is always *perpendicular* (orthogonal) to the level curve passing through  $\mathbf{x}$ . Intuitively: if you stand on a hillside and look along a contour line, the steepest direction is exactly at right angles to it.

Moving in the direction *opposite* the gradient — i.e., in the direction  $-\nabla f(\mathbf{x})$  — is the direction of steepest descent. Gradient-based minimizers update the current point  $\mathbf{x}$  as:

$$\mathbf{x}_{\text{new}} = \mathbf{x} - \alpha \nabla f(\mathbf{x})$$

where  $\alpha$  (alpha) is a positive step size called the *learning rate*. We will revisit this update rule in

# The Jacobian Matrix I

When a function has *multiple outputs* — not just a single number but a vector of numbers — the natural generalization of the gradient is the **Jacobian matrix**.

## Definition: Jacobian

For a function  $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$  that maps an  $n$ -dimensional input vector to an  $m$ -dimensional output vector, the **Jacobian  $\mathbf{J}$**  is the  $m \times n$  matrix whose  $(i, j)$ -th entry is:

$$\mathbf{J}_{ij} = \frac{\partial f_i}{\partial x_j}, \quad \mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

# The Jacobian Matrix II

## Notation

- $f_i$  is the  $i$ -th component of the output vector  $\mathbf{f}(\mathbf{x})$ , for  $i = 1, \dots, m$ .
- $x_j$  is the  $j$ -th component of the input vector  $\mathbf{x}$ , for  $j = 1, \dots, n$ .
- $\mathbf{J}_{ij}$  (row  $i$ , column  $j$ ) measures how the  $i$ -th output changes per unit change in the  $j$ -th input.
- “ $m \times n$  matrix” means  $m$  rows and  $n$  columns.

# The Jacobian Matrix III

## Special cases that connect to familiar objects:

- When  $m = 1$  (scalar output),  $\mathbf{f}$  is just an ordinary scalar function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$ , and the Jacobian reduces to a  $1 \times n$  row vector equal to  $\nabla f(\mathbf{x})^\top$ . So the gradient is just the Jacobian transposed.
- When  $n = 1$  (scalar input), the Jacobian is an  $m \times 1$  column vector of ordinary scalar derivatives.

**Chain rule for vector functions.** Just as in one dimension, composed functions obey a chain rule. If we have  $\mathbf{h}(\mathbf{x}) = \mathbf{f}(\mathbf{g}(\mathbf{x}))$ , then the Jacobian of the composition is the product of the Jacobians:

$$\mathbf{J}_h = \mathbf{J}_f \cdot \mathbf{J}_g.$$

This matrix multiplication is the multi-dimensional generalization of multiplying scalar derivatives when applying the chain rule. It is the core identity underlying automatic differentiation.

## The Jacobian Matrix IV

### Worked Example

Let  $\mathbf{f}(u, v) = (u + v, uv)^\top$  and  $\mathbf{g}(x) = (x^2, x^3)^\top$ . The Jacobian of  $\mathbf{g}$  at  $x = 2$  is  $\mathbf{J}_{\mathbf{g}} = \begin{bmatrix} 2x \\ 3x^2 \end{bmatrix} \big|_{x=2} = \begin{bmatrix} 4 \\ 12 \end{bmatrix}$ . The Jacobian of  $\mathbf{f}$  at  $(u, v) = (4, 8)$  is  $\mathbf{J}_{\mathbf{f}} = \begin{bmatrix} 1 & 1 \\ v & u \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 8 & 4 \end{bmatrix}$ . By the chain rule:  $\mathbf{J}_{\mathbf{h}} = \begin{bmatrix} 1 & 1 \\ 8 & 4 \end{bmatrix} \begin{bmatrix} 4 \\ 12 \end{bmatrix} = \begin{bmatrix} 16 \\ 64 \end{bmatrix}$ .

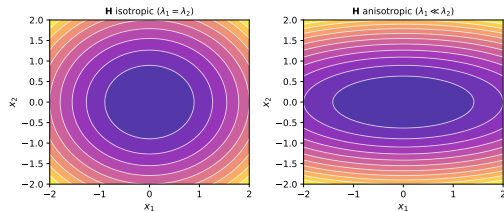
# The Hessian Matrix: Capturing Curvature I

The gradient captures the *first-order* (linear) behavior of  $f$  near a point. For a better approximation — and for understanding whether a critical point is a minimum, maximum, or saddle — we need the *second-order* (quadratic) information. This is encoded by the **Hessian matrix**.

## Definition: Hessian (eq. 2.8)

For a function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$ , the **Hessian  $\mathbf{H}$**  is the symmetric  $n \times n$  matrix of all second-order partial derivatives:

$$\mathbf{H}_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$
$$\mathbf{H} = \begin{bmatrix} f_{x_1 x_1} & \cdots & f_{x_1 x_n} \\ \vdots & \ddots & \vdots \\ f_{x_n x_1} & \cdots & f_{x_n x_n} \end{bmatrix}$$



Top: isotropic curvature ( $\lambda_1 = \lambda_2$ , circular contours).

Bottom: anisotropic curvature — the function curves much more steeply in one direction, producing elongated elliptical contours.

# The Hessian Matrix: Capturing Curvature II

## Notation

- $\frac{\partial^2 f}{\partial x_i \partial x_j}$  (“second partial of  $f$  with respect to  $x_i$  then  $x_j$ ”) means: first differentiate  $f$  with respect to  $x_j$  (holding all other variables fixed), then differentiate the result with respect to  $x_i$ .
- $f_{x_i x_j}$  is a compact shorthand for the same quantity.
- The diagonal entry  $\mathbf{H}_{ii} = \partial^2 f / \partial x_i^2$  is the ordinary second derivative with respect to  $x_i$ : it measures how the slope in the  $x_i$  direction is itself changing.
- $\lambda$  (lambda) will denote eigenvalues of  $\mathbf{H}$  in second-order analysis.

**Symmetry.** Under mild smoothness conditions (Schwarz’s theorem), the order of differentiation does not matter:  $\mathbf{H}_{ij} = \mathbf{H}_{ji}$ . So the Hessian is always a *symmetric* matrix.

**What the Hessian tells an optimizer.** The sign of the Hessian (more precisely, whether it is *positive definite*) classifies critical points: if  $\nabla f(\mathbf{x}) = \mathbf{0}$  and  $\mathbf{H}$  is positive definite (all eigenvalues  $> 0$ ), then  $\mathbf{x}$  is a local minimum. If  $\mathbf{H}$  is negative definite (all eigenvalues  $< 0$ ),  $\mathbf{x}$  is a local maximum. If the eigenvalues have mixed signs,  $\mathbf{x}$  is a saddle point.



# The Hessian Matrix: Capturing Curvature III

**The second-order Taylor approximation** of  $f$  near  $\mathbf{x}$ :

$$f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \Delta\mathbf{x} + \frac{1}{2} \Delta\mathbf{x}^\top \mathbf{H}(\mathbf{x}) \Delta\mathbf{x}$$

The last term  $\frac{1}{2} \Delta\mathbf{x}^\top \mathbf{H}(\mathbf{x}) \Delta\mathbf{x}$  is a *quadratic form* (a generalization of  $\frac{1}{2}f''(x)h^2$  in 1-D) that captures the curvature.

## Worked Example

Let  $f(x_1, x_2) = x_1^2 + 3x_1x_2 + 2x_2^2$ . The first partial derivatives are  $\partial f / \partial x_1 = 2x_1 + 3x_2$  and  $\partial f / \partial x_2 = 3x_1 + 4x_2$ . The second partial derivatives are  $f_{x_1x_1} = 2$ ,  $f_{x_1x_2} = f_{x_2x_1} = 3$ ,  $f_{x_2x_2} = 4$ . So the Hessian is the constant matrix  $\mathbf{H} = \begin{bmatrix} 2 & 3 \\ 3 & 4 \end{bmatrix}$ , which is symmetric.

# Directional Derivative I

The gradient tells us the rate of change along each coordinate axis. But what if we want to know the rate of change along an *arbitrary direction*? That is the purpose of the **directional derivative**.

## Definition: Directional Derivative (eq. 2.10)

The **directional derivative** of  $f$  at  $\mathbf{x}$  in the direction of a vector  $\mathbf{s} \in \mathbb{R}^n$  is:

$$\nabla_{\mathbf{s}} f(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{s}) - f(\mathbf{x})}{h} = \nabla f(\mathbf{x})^\top \mathbf{s}$$

## Notation

- $\mathbf{s}$  (“ $\mathbf{s}$  bold”) is any nonzero direction vector. It is common to require  $\|\mathbf{s}\| = 1$  (unit length) so that the directional derivative equals the actual slope (not scaled by the length of  $\mathbf{s}$ ).
- $\nabla f(\mathbf{x})^\top \mathbf{s}$  is the *dot product* (inner product) of the gradient vector and the direction vector. For two vectors  $\mathbf{a}$  and  $\mathbf{b}$ ,  $\mathbf{a}^\top \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$  where  $\theta$  (theta) is the angle between them.
- $\mathbf{e}_i$  denotes the  $i$ -th standard basis vector (all zeros except a 1 in position  $i$ ).

# Directional Derivative II

## How to Read This Formula

Because  $\nabla f(\mathbf{x})^\top \mathbf{s} = \|\nabla f(\mathbf{x})\| \|\mathbf{s}\| \cos \theta$ , the directional derivative is largest when  $\theta = 0$  (i.e.,  $\mathbf{s}$  points in the same direction as  $\nabla f(\mathbf{x})$ ) and smallest (most negative) when  $\theta = \pi$  (opposite direction). This confirms that the gradient itself is the direction of steepest ascent.

# Directional Derivative III

**Special cases.** If we choose  $\mathbf{s} = \mathbf{e}_i$  (the  $i$ -th unit coordinate vector), then the directional derivative reduces to:

$$\nabla_{\mathbf{e}_i} f(\mathbf{x}) = \nabla f(\mathbf{x})^\top \mathbf{e}_i = \frac{\partial f}{\partial x_i}.$$

So ordinary partial derivatives are just directional derivatives along coordinate axes. The directional derivative is the unified concept.

## Directional Derivative IV

### Worked Numerical Example

Let  $f(x_1, x_2) = x_1^2 + x_2^2$ , evaluated at  $\mathbf{x} = (1, 1)^\top$ . Choose direction  $\mathbf{s} = (\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})^\top$  (the diagonal direction, which has unit length since  $\|\mathbf{s}\| = \sqrt{(1/\sqrt{2})^2 + (1/\sqrt{2})^2} = 1$ ).

Step 1: Compute the gradient.  $\nabla f(x_1, x_2) = (2x_1, 2x_2)^\top$ , so  $\nabla f(1, 1) = (2, 2)^\top$ .

Step 2: Compute the dot product.

$$\nabla_{\mathbf{s}} f = (2, 2) \cdot \left( \frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right) = 2 \cdot \frac{1}{\sqrt{2}} + 2 \cdot \frac{1}{\sqrt{2}} = \frac{2}{\sqrt{2}} + \frac{2}{\sqrt{2}} = 2\sqrt{2} \approx 2.83.$$

This means: if we move in the diagonal direction from  $(1, 1)$ ,  $f$  increases at a rate of approximately 2.83 per unit distance. For comparison, moving along the  $x_1$ -axis gives a rate of  $\partial f / \partial x_1 = 2$ , which is smaller.

# Finite Difference Methods: Overview and Intuition I

When a function  $f$  is available only as a piece of computer code — not as an algebraic formula — we cannot apply the rules of calculus directly. Instead, we can *approximate* the derivative by evaluating  $f$  at nearby points and measuring how much the output changes. This family of approaches is called **finite difference approximation**.

The idea is simply to replace the theoretical limit in the definition of the derivative with a small but nonzero step  $h$ : instead of taking  $h \rightarrow 0$ , we just pick a small value of  $h$  and compute the ratio. The error we introduce by not going all the way to zero is called the *truncation error*.

## The Three Finite Difference Approximations

| Method              | Formula                         | Truncation Error                       |
|---------------------|---------------------------------|----------------------------------------|
| Forward difference  | $\frac{f(x+h) - f(x)}{h}$       | $O(h)$ — error proportional to $h$     |
| Central difference  | $\frac{f(x+h/2) - f(x-h/2)}{h}$ | $O(h^2)$ — error proportional to $h^2$ |
| Backward difference | $\frac{f(x) - f(x-h)}{h}$       | $O(h)$ — error proportional to $h$     |

## How to Read the Error Column

“ $O(h)$ ” (pronounced “big-O of  $h$ ”) means the error is roughly proportional to  $h$ : halving  $h$  halves the error. “ $O(h^2)$ ” means the error is proportional to  $h^2$ : halving  $h$  *quarters* the error. So central difference converges much faster as  $h$  decreases — it is second-order accurate while forward and backward are only first-order accurate.



# Forward Difference: Taylor Series Analysis I

To understand *why* the forward difference approximation has an  $O(h)$  error, we use the **Taylor series** expansion of  $f$  around the point  $x$ .

## Taylor Expansion of $f(x + h)$ around $x$

For any smooth function  $f$ :

$$f(x + h) = f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \dots$$

## Notation

- $f''(x)$  (“f double-prime of x”) is the second derivative of  $f$  at  $x$ : it measures the rate of change of the slope.
- $f'''(x)$  (“f triple-prime of x”) is the third derivative.
- $2! = 2 \times 1 = 2$  and  $3! = 3 \times 2 \times 1 = 6$  are factorial numbers (the “!” symbol means factorial).
- The dots ( $\dots$ ) indicate higher-order terms that become negligible as  $h \rightarrow 0$ .

## Forward Difference: Taylor Series Analysis II

Rearranging the expansion to isolate  $f'(x)$ :

$$f'(x) = \frac{f(x+h) - f(x)}{h} - \underbrace{\frac{f''(x)}{2} h - \frac{f'''(x)}{6} h^2 - \dots}_{\text{truncation error}}$$

The forward difference formula drops all the error terms, keeping only the first fraction. The dominant error term is  $-\frac{f''(x)}{2} h$ , which is proportional to  $h$ . This confirms the  $O(h)$  accuracy.

# Forward Difference: Taylor Series Analysis III

**The step-size dilemma.** You might think: simply make  $h$  very small and the error disappears. Unfortunately, there is a competing source of error: *floating-point round-off*. When  $h$  is so small that  $f(x+h)$  and  $f(x)$  are nearly equal, their difference  $f(x+h) - f(x)$  is computed by subtracting two nearly identical numbers. In floating-point arithmetic, this causes *subtractive cancellation*: the significant digits cancel and the result is dominated by rounding error.

## Two competing errors:

- **Truncation error** ( $\propto h$ ): decreases as  $h$  shrinks.
- **Round-off error** ( $\propto 1/h$  approximately): grows as  $h$  shrinks.

There is an *optimal*  $h$  that balances the two.

**Practical recommendation:** For forward difference in 64-bit floating-point arithmetic (where the machine epsilon  $\epsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$ ), the optimal step size is approximately:

$$h \approx \sqrt{\epsilon_{\text{mach}}} \approx 10^{-8}$$

Using  $h$  much smaller than this will *increase* the total error due to cancellation.

# Central Difference: Why It Is More Accurate I

The central difference formula uses symmetric points on both sides of  $x$  and achieves  $O(h^2)$  accuracy rather than  $O(h)$ . Here is why.

We expand both  $f(x + h/2)$  and  $f(x - h/2)$  using Taylor's theorem:

## Two Taylor Expansions

$$f\left(x + \frac{h}{2}\right) = f(x) + f'(x)\frac{h}{2} + \frac{f''(x)}{2}\left(\frac{h}{2}\right)^2 + \frac{f'''(x)}{6}\left(\frac{h}{2}\right)^3 + \dots$$

$$f\left(x - \frac{h}{2}\right) = f(x) - f'(x)\frac{h}{2} + \frac{f''(x)}{2}\left(\frac{h}{2}\right)^2 - \frac{f'''(x)}{6}\left(\frac{h}{2}\right)^3 + \dots$$

Now subtract the second equation from the first. Notice that every term with an *even power of  $h$*  cancels (because the  $(\cdot)^2$  terms are identical), and every term with an *odd power of  $h$*  survives but with coefficient doubled:

$$f\left(x + \frac{h}{2}\right) - f\left(x - \frac{h}{2}\right) = f'(x)h + \frac{f'''(x)}{6} \cdot \frac{h^3}{4} + \dots$$

# Central Difference: Why It Is More Accurate II

Dividing by  $h$ :

$$f'(x) = \frac{f(x + h/2) - f(x - h/2)}{h} - \frac{f'''(x)}{24} h^2 + \dots$$

The leading error term is now  $O(h^2)$ : the even-order (second-derivative) term has cancelled out exactly, leaving only the third-derivative term.

## Key Insight

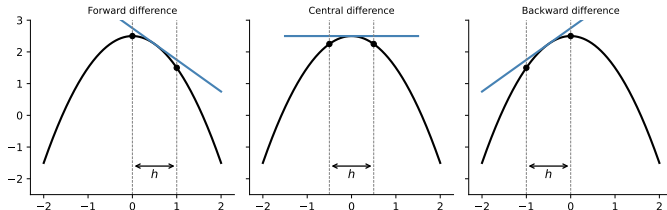
The cancellation of even-order terms is the reason central differences are more accurate. The error drops as  $h^2$  rather than  $h$ , so reducing  $h$  by a factor of 10 reduces the error by a factor of 100. The optimal step size for central difference is approximately  $h \approx \epsilon_{\text{mach}}^{1/3} \approx 10^{-5}$ .

# Finite Difference: Python Implementation

```
1 import numpy as np
2
3 def diff_forward(f, x, h=1e-8):
4     """Forward difference:  $O(h)$  error. Use  $h \sim \sqrt{\text{eps\_machine}}$ ."""
5     return (f(x + h) - f(x)) / h
6
7 def diff_central(f, x, h=1e-5):
8     """Central difference:  $O(h^2)$  error. Use  $h \sim \text{eps\_machine}^{1/3}$ ."""
9     return (f(x + h/2) - f(x - h/2)) / h
10
11 def diff_backward(f, x, h=1e-8):
12     """Backward difference:  $O(h)$  error."""
13     return (f(x) - f(x - h)) / h
14
15 # Gradient for multivariate  $f: \mathbb{R}^n \rightarrow \mathbb{R}$  (central difference per component)
16 def gradient_fd(f, x, h=1e-5):
17     """Numerical gradient using central differences along each axis."""
18     n = len(x)
19     g = np.zeros(n)
20     for i in range(n):
21         xp, xm = x.copy(), x.copy()
22         xp[i] += h/2; xm[i] -= h/2
23         g[i] = (f(xp) - f(xm)) / h
24     return g
25
26 # Running example:  $f(x) = \sin(x^2)$ ,  $x = \pi/2$ 
27 f = lambda x: np.sin(x**2)
```

# Finite Difference: Visual Comparison I

The figure below illustrates how the three finite difference methods relate geometrically to the true tangent.



**Reading the figure.** All three methods approximate the tangent slope (the true derivative, shown as the blue tangent line) by the slope of a secant line connecting two points on the curve:

- **Forward difference** connects  $(x, f(x))$  and  $(x + h, f(x + h))$  — a secant leaning to the right.
- **Backward difference** connects  $(x - h, f(x - h))$  and  $(x, f(x))$  — a secant leaning to the left.
- **Central difference** connects  $(x - h/2, f(x - h/2))$  and  $(x + h/2, f(x + h/2))$  — a symmetric secant centered on  $x$ . Because it is symmetric, it inherits the same slope error cancellation we derived above, giving  $O(h^2)$  accuracy.

# The Complex-Step Method: Avoiding Cancellation I

Finite differences suffer from a fundamental tension: smaller  $h$  reduces the truncation error but amplifies the round-off error from subtractive cancellation. The **complex-step method** cleverly eliminates this cancellation entirely by evaluating  $f$  at a *complex* argument.

## Core Idea

Instead of comparing two real evaluations, evaluate  $f$  at the complex point  $x + ih$ , where  $i = \sqrt{-1}$  is the imaginary unit and  $h$  is a small real number.

Expanding  $f(x + ih)$  in a Taylor series (treating  $ih$  as the small perturbation):

$$f(x + ih) = f(x) + ih f'(x) + \frac{(ih)^2}{2!} f''(x) + \frac{(ih)^3}{3!} f'''(x) + \dots$$

Using  $i^2 = -1$  and  $i^3 = -i$ :

$$f(x + ih) = \underbrace{f(x) - \frac{h^2}{2} f''(x) + \dots}_{\text{Real part}} + i \underbrace{\left( h f'(x) - \frac{h^3}{6} f'''(x) + \dots \right)}_{\text{Imaginary part}}$$



## The Complex-Step Method: Avoiding Cancellation II

Taking the imaginary part and dividing by  $h$ :

$$f'(x) \approx \frac{\operatorname{Im}(f(x + ih))}{h} + O(h^2)$$

# The Complex-Step Method: Avoiding Cancellation III

## Notation

- $i$  is the imaginary unit satisfying  $i^2 = -1$ . A complex number is any number of the form  $a + bi$  where  $a$  and  $b$  are real numbers.
- $\text{Im}(\cdot)$  (“imaginary part of”) extracts the  $b$  from  $a + bi$ .
- $\text{Re}(\cdot)$  (“real part of”) extracts  $a$ .

## Why There Is No Cancellation

In finite differences we compute  $f(x + h) - f(x)$  — a subtraction of two nearly equal numbers, which destroys precision. In the complex-step method we extract the *imaginary part* of a single function evaluation. Imaginary and real parts do not cancel each other, so there is no subtractive cancellation. This means we can safely use  $h = 10^{-20}$  or even smaller, obtaining near-machine-precision derivatives.

**Requirement.** The function  $f$  must be able to accept a complex number as input and produce a complex number as output. This is called being *analytic* (or holomorphic). Most mathematical

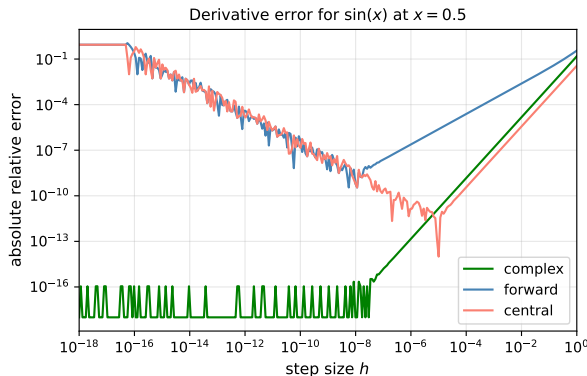
# The Complex-Step Method: Avoiding Cancellation IV

functions (polynomials,  $\sin$ ,  $\exp$ ,  $\ln$ , etc.) are analytic. NumPy handles complex arrays transparently, so Python code written with NumPy usually works without modification.

# Complex-Step Method: Python Code and Example

```
1 import numpy as np
2
3 def diff_complex(f, x, h=1e-20):
4     """
5     Complex-step derivative:  $\text{Im}(f(x + i*h)) / h$ 
6     No subtractive cancellation:  $h$  can be made arbitrarily small.
7     """
8     return np.imag(f(x + h * 1j)) / h
9
10 # Example:  $f(x) = \sin(x^2)$ 
11 # True derivative by chain rule:  $f'(x) = 2x * \cos(x^2)$ 
12 f = lambda x: np.sin(x**2)
13 x0 = np.pi / 2
14
15 # A single complex evaluation gives BOTH  $f(x_0)$  and  $f'(x_0)$ 
16 v = f(x0 + 1e-20j)
17 fx = np.real(v) #  $f(x_0) \sim 0.62426$ 
18 dfdx = np.imag(v) / 1e-20 #  $f'(x_0)$ 
19
20 print(f"f(pi/2) = {fx:.8f}")
21 print(f"f'(pi/2) = {dfdx:.8f}")
22
23 # True derivative:  $2*(\pi/2)*\cos((\pi/2)^2) = \pi * \cos(\pi^2/4)$ 
24 true_d = np.pi * np.cos(np.pi**2 / 4)
25 print(f"True = {true_d:.8f}")
26 # Both complex-step and true agree to all 8 decimal places shown.
```

# Error Comparison: Forward vs. Central vs. Complex-Step I



## Reading the log-log error plot.

The horizontal axis shows the step size  $h$  on a logarithmic scale (each tick is a power of 10), and the vertical axis shows the absolute error in the derivative estimate, also on a logarithmic scale.

**Forward difference** (blue): the error decreases linearly (slope  $-1$  on a log-log plot, confirming  $O(h)$ ) until  $h$  becomes so small that round-off dominates, after which the error rises.

**Central difference** (orange): decreases with slope  $-2$  (confirming  $O(h^2)$ ), hitting a floor at a smaller error before round-off takes over.

**Complex-step** (green): stays flat near machine precision for all small  $h$  — there is no round-off floor because there is no

## Error Comparison: Forward vs. Central vs. Complex-Step II

### Key Takeaway

If your function supports complex inputs (which NumPy functions do), always prefer the complex-step method: it achieves near-machine-precision accuracy and requires exactly the same number of function evaluations (one) as forward difference, but without any step-size tuning.

# Automatic Differentiation: The Best of Both Worlds I

We have seen three approaches so far:

- **Symbolic differentiation** gives exact derivatives but requires the function to have an algebraic formula and can suffer from expression swell.
- **Finite differences** are easy to implement but approximate, with accuracy limited by step-size trade-offs.
- **Complex-step** eliminates cancellation but still requires the function to support complex inputs.

**Automatic differentiation (AD)** is a fourth approach that combines the best of symbolic and numerical methods: it computes *exact* (up to floating-point precision) derivatives of a function given as computer code, without needing a closed-form formula and without any step-size approximation.

The key insight is that any computer program is composed of a finite sequence of *elementary operations* — addition, subtraction, multiplication, division, sin, exp, ln, etc. — each of which has a known derivative. By applying the **chain rule** systematically through the sequence of operations, AD computes the derivative of the whole program.

# Automatic Differentiation: The Best of Both Worlds II

## Chain Rule (the Engine of AD)

If  $f = f(g(x))$ , then:

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg} \cdot \frac{dg}{dx}$$

## Notation

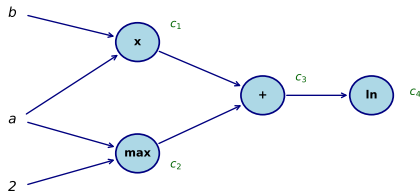
- $g(x)$  is an intermediate value computed on the way to  $f$ .
- $\frac{df}{dg}$  (“d-f over d-g”) is the derivative of the outer function with respect to its input  $g$ .
- $\frac{dg}{dx}$  (“d-g over d-x”) is the derivative of the inner function with respect to  $x$ .
- The chain rule says these two local slopes multiply to give the overall slope.



# Automatic Differentiation: The Best of Both Worlds III

**The computational graph.** Every program can be represented as a **directed acyclic graph (DAG)** where each node is an elementary operation and each edge carries a value from one operation to the next. This is called the *computational graph*.

Computational graph for  $f(a, b) = \ln(ab + \max(a, 2))$



Computational graph for  $f(a, b) = \ln(ab + \max(a, 2))$ .

The graph for  $f(a, b) = \ln(ab + \max(a, 2))$  has:

- **Leaf nodes** (inputs):  $a$ ,  $b$ , and the constant 2.
- **Internal nodes** (operations):  
 $c_1 = a \times b$ ,  $c_2 = \max(a, 2)$ ,  
 $c_3 = c_1 + c_2$ ,  $c_4 = \ln(c_3)$ .
- **Terminal node** (output):  $c_4 = f(a, b)$ .

There are two strategies to traverse this graph and accumulate derivatives: **forward accumulation** (from inputs to output) and **reverse accumulation** (from output back to inputs).

# Forward Accumulation (Forward Mode AD) I

**Idea.** Traverse the computational graph from inputs to output, computing at each node both the function value and the derivative of that intermediate value with respect to a chosen input variable. We call these derivative values *tangents* or *dots* and write  $\dot{c}_i = \partial c_i / \partial x$  for a chosen input  $x$ .

## Forward Mode Chain Rule

At each node  $c_i$  that depends on parent nodes  $c_j$ :

$$\dot{c}_i = \frac{\partial c_i}{\partial c_j} \cdot \dot{c}_j \quad (\text{accumulated over all parent nodes})$$

Seed: set  $\dot{x} = 1$  for the input variable of interest and  $\dot{y} = 0$  for all other inputs.

**Worked Example (eq. 2.26–2.30).** Compute  $\partial f / \partial a$  for  $f(a, b) = \ln(ab + \max(a, 2))$  at  $a = 3$ ,  $b = 2$ . Seed:  $\dot{a} = 1$ ,  $\dot{b} = 0$ .

| Node               | Value | Derivative $\dot{c}$      | Why          | Result:                                             |                                                                      |  |
|--------------------|-------|---------------------------|--------------|-----------------------------------------------------|----------------------------------------------------------------------|--|
| $a$                | 3     | $\dot{a} = 1$             | seed         | $f(3, 2) = \ln 9 \approx 2.197$                     | $\left. \frac{\partial f}{\partial a} \right _{(3,2)} = \frac{1}{3}$ |  |
| $b$                | 2     | $\dot{b} = 0$             | seed         |                                                     |                                                                      |  |
| $c_1 = ab$         | 6     | $b\dot{a} + a\dot{b} = 2$ | product rule | <b>Cost:</b> One forward pass computes the value of |                                                                      |  |
| $c_2 = \max(a, 2)$ | 3     | $\dot{c}_2 = 1$           | $a > 2$      |                                                     |                                                                      |  |

## Forward Accumulation (Forward Mode AD) II

**Dual numbers: an elegant implementation of forward mode.**

A **dual number** is a pair  $(a, b)$  written as  $a + b\epsilon$  (epsilon), where  $\epsilon$  (“epsilon”) is an abstract symbol with the special property  $\epsilon^2 = 0$  (but  $\epsilon \neq 0$ ).

The value  $a$  stores the function value, and  $b$  stores the derivative. Dual number arithmetic automatically propagates both simultaneously:

### Dual Number Arithmetic (eq. 2.32–2.33)

$$(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon$$

$$(a + b\epsilon) \times (c + d\epsilon) = (ac) + (ad + bc)\epsilon + \underbrace{bd\epsilon^2}_{=0} = (ac) + (ad + bc)\epsilon$$

The multiplication rule  $(ad + bc)$  is exactly the product rule from calculus: if  $c_1 = a \cdot b$  then  $\dot{c}_1 = \dot{a}b + a\dot{b}$ . So dual number arithmetic encodes the chain rule for free.

To compute  $f'(x_0)$ : initialize  $x = x_0 + 1 \cdot \epsilon$  and run  $f$  normally. The  $\epsilon$ -coefficient of the output is  $f'(x_0)$ .

# Forward AD: Python Implementation with Dual Numbers

```
1 import numpy as np
2
3 class Dual:
4     """A dual number  $v + d \cdot \epsilon$ , where  $\epsilon^2 = 0$ .
5      $v$  holds the function value;  $d$  holds the derivative."""
6     def __init__(self, v, d=0.0):
7         self.v, self.d = float(v), float(d)
8
9     def __add__(self, other):
10        if isinstance(other, Dual):
11            return Dual(self.v + other.v, self.d + other.d)
12        return Dual(self.v + other, self.d)          # add scalar constant
13    __radd__ = __add__
14
15    def __mul__(self, other):
16        if isinstance(other, Dual):
17            # product rule:  $(a+b\epsilon)(c+d\epsilon) = ac + (ad+bc)\epsilon$ 
18            return Dual(self.v * other.v,
19                        self.v * other.d + self.d * other.v)
20        return Dual(self.v * other, self.d * other)
21    __rmul__ = __mul__
22
23    def log(self):
24        #  $d/dx \ln(x) = 1/x$ , so chain rule gives: derivative =  $self.d / self.v$ 
25        return Dual(np.log(self.v), self.d / self.v)
26
27 # Example:  $f(a,b) = \log(a \cdot b + \max(a, 2))$  at  $a=3, b=2$ 
```

# Reverse Accumulation (Backpropagation) I

While forward mode computes derivatives one input at a time, **reverse mode** computes derivatives with respect to *all inputs simultaneously* in a single backward pass. This makes it the method of choice when there are many inputs and few outputs — the standard situation in machine learning, where  $f$  is a scalar loss function of millions of parameters.

**The idea.** Define the **adjoint** (sensitivity) of each intermediate value  $c_i$  as:

$$\bar{c}_i = \frac{\partial f}{\partial c_i}$$

This measures how much the final output  $f$  would change if  $c_i$  changed by a small amount. The output node has adjoint  $\bar{c}_{\text{output}} = 1$  (changing the output by 1 changes  $f$  by 1).

# Reverse Accumulation (Backpropagation) II

## Reverse Mode Propagation Rule

For each node  $c_i$  with children  $c_j$  (nodes that use  $c_i$ ):

$$\bar{c}_i = \sum_{j: j \text{ uses } i} \bar{c}_j \cdot \frac{\partial c_j}{\partial c_i}$$

## Notation

- $\bar{c}_i$  (“c-bar sub-i”, read as “adjoint of  $c_i$ ”) is the sensitivity of the output to the intermediate value  $c_i$ .
- The sum runs over all nodes  $c_j$  that directly depend on  $c_i$  in the computational graph.
- $\frac{\partial c_j}{\partial c_i}$  is the local derivative of the operation at node  $c_j$  with respect to its input  $c_i$ .

# Reverse Accumulation (Backpropagation) III

## Two-phase procedure.

- ➊ **Forward pass:** run the program normally to compute all intermediate values  $c_i$  and the output  $f$ . Store (“tape”) all intermediate values needed for the backward pass.
- ➋ **Backward pass:** propagate adjoints from the output back to the inputs, applying the rule above at each node.

The backward pass costs approximately 2–3 times a single forward pass, regardless of the number of inputs  $n$ . So the total cost is about  $3\times$  a forward evaluation to get the *full gradient* with respect to all  $n$  inputs simultaneously. For neural networks with millions of parameters, this is an enormous saving over forward mode (which would need  $n$  forward passes).

# Reverse Accumulation (Backpropagation) IV

## Forward vs. Reverse Mode Comparison

|               | Forward mode           | Reverse mode (backprop)                  |
|---------------|------------------------|------------------------------------------|
| Passes needed | $n$ (one per input)    | $m$ (one per output)                     |
| Best when     | $n \ll m$ (few inputs) | $m \ll n$ (few outputs)                  |
| Memory cost   | $O(1)$                 | $O(\text{graph size})$ — must store tape |
| Deep learning | less common            | <b>this is backpropagation</b>           |

**The deep learning connection.** In deep learning, the function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  is the scalar training loss and  $n$  is the number of model parameters (potentially billions). Reverse mode (backpropagation) computes the full gradient  $\nabla f$  in a single backward pass — this is why training large neural networks is computationally feasible at all.



# Reverse AD in Practice: Using JAX

```
1 # Install: pip install jax
2 import jax
3 import jax.numpy as jnp
4
5 # Define a function with two inputs
6 def f(x):
7     #  $f(a, b) = \ln(a*b + \max(a, 2.0))$ 
8     return jnp.log(x[0]*x[1] + jnp.maximum(x[0], 2.0))
9
10 x0 = jnp.array([3.0, 2.0])
11
12 # --- Reverse mode: full gradient in one backward pass ---
13 grad_f = jax.grad(f)           # grad_f is a function that returns the gradient
14 g = grad_f(x0)                 # [df/da, df/db]
15 print(f"df/da = {g[0]:.4f}")   # expected: 0.3333
16 print(f"df/db = {g[1]:.4f}")   # expected: 0.3333
17
18 # --- Forward mode: Jacobian-vector product ---
19 # jax.jvp computes (f(x0), J(x0) * v) where v is a tangent vector
20 primals, tangents = jax.jvp(f, (x0,), (jnp.array([1.0, 0.0]),))
21 print(f"f(3,2) = {primals:.4f}") # ~2.1972
22 print(f"df/da via JVP = {tangents:.4f}") # 0.3333
23
24 # --- Hessian via nested AD ---
25 hess = jax.hessian(f)(x0)
26 print("Hessian:\n", hess)
```

# Regression Gradient: Estimating Gradients for Black-Box Functions I

All the methods discussed so far — symbolic, finite difference, complex-step, and automatic differentiation — require some form of access to the internals of  $f$ : either its algebraic formula (symbolic, AD) or the ability to evaluate it at complex arguments (complex-step).

**What if  $f$  is a true black box?** For example,  $f$  might be a physical simulation, an experiment, or a proprietary software tool that can only be queried at real-valued inputs. In that case, we can estimate the gradient using only real-valued function evaluations, by treating gradient estimation as a *linear regression problem*.

**The idea.** Near any point  $\mathbf{x}$ , the first-order Taylor approximation says:

$$f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + \mathbf{g}^\top \Delta\mathbf{x}$$

where  $\mathbf{g}$  (“ $\mathbf{g}$  bold”) is the gradient we want to estimate. If we evaluate  $f$  at  $m$  perturbed points  $\mathbf{x} + \Delta\mathbf{x}^{(1)}, \dots, \mathbf{x} + \Delta\mathbf{x}^{(m)}$ , we get  $m$  approximate equations of this form. We can then find the  $\mathbf{g}$  that best fits all these equations simultaneously — that is, solve a least-squares regression problem.

# Regression Gradient: Estimating Gradients for Black-Box Functions II

## The Optimization Problem (eq. 2.42)

Find the vector  $\mathbf{g} \in \mathbb{R}^n$  minimizing the sum of squared fitting errors:

$$\min_{\mathbf{g}} \sum_{i=1}^m \left( f(\mathbf{x} + \Delta\mathbf{x}^{(i)}) - \left( f(\mathbf{x}) + \mathbf{g}^\top \Delta\mathbf{x}^{(i)} \right) \right)^2$$

## Notation

- $\Delta\mathbf{x}^{(i)}$  (“Delta x superscript i”; the superscript in parentheses is an *index*, not a power) is the  $i$ -th perturbation vector: the direction and distance we moved  $\mathbf{x}$  to get the  $i$ -th sample point.
- $m$  is the number of perturbation samples.
- $\sum_{i=1}^m$  (Sigma, “sum from  $i$  equals one to  $m$ ”) means we add up the squared errors for all  $m$  perturbations.
- $\mathbf{g}^\top \Delta\mathbf{x}^{(i)}$  is the predicted change in  $f$  according to the linear model.

# Regression Gradient: Estimating Gradients for Black-Box Functions III

## Matrix Formulation and Closed-Form Solution (eq. 2.43–2.46)

Stack the perturbations into a matrix and the function differences into a vector:

$$\Delta \mathbf{X} = \begin{bmatrix} (\Delta \mathbf{x}^{(1)})^\top \\ \vdots \\ (\Delta \mathbf{x}^{(m)})^\top \end{bmatrix} \in \mathbb{R}^{m \times n}, \quad \Delta \mathbf{f} = \begin{bmatrix} f(\mathbf{x} + \Delta \mathbf{x}^{(1)}) - f(\mathbf{x}) \\ \vdots \\ f(\mathbf{x} + \Delta \mathbf{x}^{(m)}) - f(\mathbf{x}) \end{bmatrix} \in \mathbb{R}^m$$

The minimization problem becomes a standard least-squares problem:  $\min_{\mathbf{g}} \|\Delta \mathbf{X} \mathbf{g} - \Delta \mathbf{f}\|^2$ . The closed-form solution is:

$$\mathbf{g} = \Delta \mathbf{X}^+ \Delta \mathbf{f}$$

where  $\Delta \mathbf{X}^+$  (“Delta X pseudoinverse”) is the **Moore-Penrose pseudoinverse** of  $\Delta \mathbf{X}$ .

# Regression Gradient: Estimating Gradients for Black-Box Functions IV

## How to Read This Formula

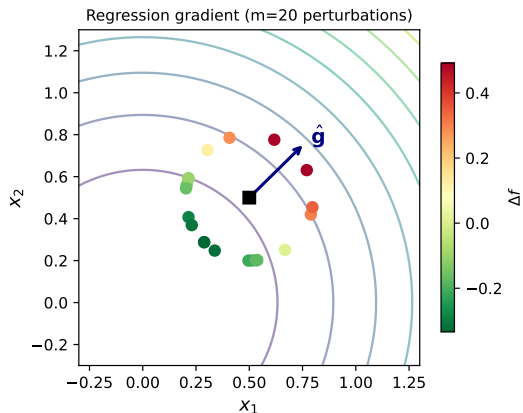
The pseudoinverse  $\Delta\mathbf{X}^+$  is a generalization of the ordinary matrix inverse that works even when  $\Delta\mathbf{X}$  is not square (we typically have  $m > n$  so there are more equations than unknowns). It finds the least-squares solution automatically. In practice, use `np.linalg.lstsq` rather than explicitly computing the pseudoinverse.

**Generating perturbations.** Draw each  $\Delta\mathbf{x}^{(i)}$  uniformly from a hypersphere of radius  $\delta$  (delta) centered at the origin. This ensures the perturbations uniformly cover all directions so the regression is well-conditioned.

# Regression Gradient: Python Implementation

```
1 import numpy as np
2
3 def regression_gradient(f, x, m=20, delta=0.1):
4     """
5     Estimate gradient of black-box f at point x using
6     m function evaluations at random perturbations of radius delta.
7     Returns the least-squares gradient estimate g in  $\mathbb{R}^n$ .
8     """
9     n = len(x)
10    fx = f(x)
11    # Step 1: draw m random unit-direction vectors and scale by delta
12    dX = np.random.randn(m, n) # shape (m, n)
13    dX /= np.linalg.norm(dX, axis=1, keepdims=True) # normalize to unit sphere
14    dX *= delta # scale to desired radius
15
16    # Step 2: evaluate function differences at each perturbed point
17    df = np.array([f(x + dx) - fx for dx in dX]) # shape (m,)
18
19    # Step 3: solve least-squares: dX @ g = df
20    g, *_ = np.linalg.lstsq(dX, df, rcond=None)
21    return g
22
23 # Verification: f(x) = x1^2 + x2^2, true gradient at (1,1) is (2, 2)
24 f = lambda x: x[0]**2 + x[1]**2
25 x0 = np.array([1.0, 1.0])
26 np.random.seed(42)
27 g_est = regression_gradient(f, x0, m=50, delta=0.01)
```

# Regression Gradient: Geometric Interpretation and Limitations I



**Reading the figure.** Each dot is one perturbation  $\mathbf{x} + \Delta\mathbf{x}^{(i)}$ , colored by whether  $f$  increased (red) or decreased (green) relative to  $f(\mathbf{x})$ . The regression fit finds the linear trend in these colors, and the direction of fastest increase is the estimated gradient (navy arrow). Points in the direction of the true gradient tend to be red; points in the opposite direction tend to be green.

Sample perturbations colored by  $\Delta f$  value. Navy arrow: estimated gradient direction. More perturbations yield a more accurate estimate.

# Regression Gradient: Geometric Interpretation and Limitations II

## Strengths:

- Works for any black-box function, including non-differentiable or discontinuous functions (though the gradient estimate is then an approximation to a sub-gradient).
- Works even when  $f$  is noisy (stochastic), because the least-squares fit averages out noise.
- Requires only  $m + 1$  function evaluations (including  $f(\mathbf{x})$  itself).

## Limitations:

- Requires  $m \geq n$  evaluations to have an overdetermined system (more equations than unknowns), which becomes expensive for large  $n$ .
- Accuracy degrades in high dimensions — with  $m$  fixed, each individual direction is sampled less densely.
- For very large  $n$ , SPSA (next section) is far more efficient.



# Regression Gradient: Geometric Interpretation and Limitations III

## Key Takeaway

Regression gradient is a powerful tool for moderate-dimensional black-box optimization. It bridges the gap between pure gradient methods (which need differentiable  $f$ ) and derivative-free methods (which ignore gradient structure entirely).

# SPSA: Simultaneous Perturbation for Large-Scale Problems I

The regression gradient requires at least  $n + 1$  function evaluations to estimate the gradient in  $n$  dimensions. For problems with hundreds of thousands of variables — such as training a large neural network without automatic differentiation — this is prohibitively expensive.

**Simultaneous Perturbation Stochastic Approximation (SPSA)** estimates the gradient using only **2 function evaluations**, regardless of the dimension  $n$ . The price is noise: any single SPSA estimate has high variance, but the estimates are *unbiased* (they are correct on average), so averaging over many iterations converges to the true gradient.

## SPSA Core Idea

Instead of perturbing one coordinate at a time (as finite differences do), perturb *all* coordinates simultaneously with a random  $\pm 1$  vector  $\mathbf{z}$ . The ratio  $\frac{f(\mathbf{x} + \delta\mathbf{z}) - f(\mathbf{x} - \delta\mathbf{z})}{2\delta}$  measures the directional derivative along  $\mathbf{z}$ . Multiplying by  $z_i$  gives a noisy but unbiased estimate of  $\partial f / \partial x_i$ .

# SPSA: Simultaneous Perturbation for Large-Scale Problems II

## SPSA Gradient Estimate (eq. 2.48)

Draw  $\mathbf{z} \in \{-1, +1\}^n$  (a vector with each component independently  $+1$  or  $-1$  with equal probability). Then:

$$\hat{g}_i = \frac{f(\mathbf{x} + \delta \mathbf{z}) - f(\mathbf{x} - \delta \mathbf{z})}{2\delta} \cdot z_i \quad \text{for each component } i$$

## Notation

- $\delta$  (delta, a small positive number) is the perturbation radius: how far we move from  $\mathbf{x}$  in the direction  $\mathbf{z}$ . Typical values:  $\delta \in [0.01, 0.5]$  depending on the scale of  $f$ .
- $\mathbf{z} \in \{-1, +1\}^n$  is a Rademacher random vector: each component  $z_i$  (“z sub i”) is independently  $+1$  or  $-1$  with probability  $1/2$  each.
- $\hat{g}_i$  (“g-hat sub i”) is the SPSA estimate of  $\partial f / \partial x_i$ . The hat ( $\hat{\phantom{x}}$ ) denotes an estimate rather than the true value.

# SPSA: Simultaneous Perturbation for Large-Scale Problems III

**Why is  $\hat{g}_i$  an unbiased estimator of  $\partial f / \partial x_i$ ?**

The central difference formula gives approximately:

$$\frac{f(\mathbf{x} + \delta \mathbf{z}) - f(\mathbf{x} - \delta \mathbf{z})}{2\delta} \approx \nabla f(\mathbf{x})^\top \mathbf{z} = \sum_{j=1}^n \frac{\partial f}{\partial x_j} z_j$$

Multiplying by  $z_i$  and taking expectation over  $\mathbf{z}$ :

$$\mathbb{E}[\hat{g}_i] = \mathbb{E} \left[ z_i \sum_{j=1}^n \frac{\partial f}{\partial x_j} z_j \right] = \sum_{j=1}^n \frac{\partial f}{\partial x_j} \mathbb{E}[z_i z_j] = \frac{\partial f}{\partial x_i}$$

because  $\mathbb{E}[z_i z_j] = 1$  if  $i = j$  and 0 otherwise (the components of  $\mathbf{z}$  are independent).

**Using multiple samples.** Average  $m$  independent SPSA estimates to reduce variance:

$$\hat{\mathbf{g}} = \frac{1}{m} \sum_{k=1}^m \hat{\mathbf{g}}^{(k)}$$

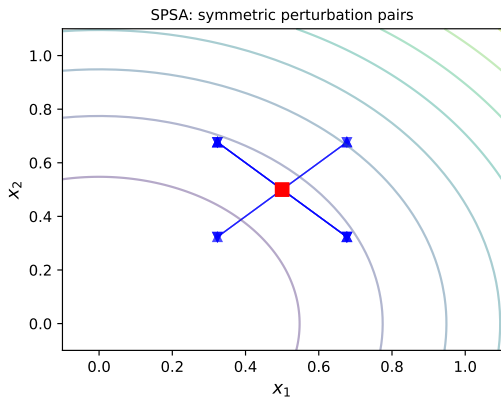
## SPSA: Simultaneous Perturbation for Large-Scale Problems IV

where  $\hat{\mathbf{g}}^{(k)}$  ("g-hat superscript  $k$ ") is the  $k$ -th estimate. The variance decreases as  $1/m$ , so using  $m = 500$  samples reduces variance by a factor of 500 at a cost of only  $2m = 1000$  function evaluations.

# SPSA: Python Implementation

```
1 import numpy as np
2
3 def spsa_gradient(f, x, delta=0.1, m=10):
4     """
5     SPSA gradient estimate using m pairs of perturbations.
6     Total cost: 2*m function evaluations (independent of dimension n).
7     For comparison, forward finite differences cost n+1 evaluations.
8     """
9     n = len(x)
10    g = np.zeros(n)
11    for _ in range(m):
12        # Draw a Rademacher vector: each component is +1 or -1 uniformly
13        z = np.random.choice([-1.0, 1.0], size=n)
14        fp = f(x + delta * z) # evaluate at x + delta*z
15        fm = f(x - delta * z) # evaluate at x - delta*z
16        # Central difference along z, then multiply by z (component-wise)
17        g += ((fp - fm) / (2 * delta)) * z
18    return g / m # average over m estimates
19
20 # Example: f(x, y) = x^2 + y^2 + 0.5*x*y
21 # True gradient at (1, 2): [2*1 + 0.5*2, 2*2 + 0.5*1] = [3.0, 4.5]
22 f = lambda x: x[0]**2 + x[1]**2 + 0.5*x[0]*x[1]
23 x0 = np.array([1.0, 2.0])
24 np.random.seed(0)
25 g_spsa = spsa_gradient(f, x0, delta=0.01, m=500)
26 print(f"SPSA estimate: {g_spsa}") # approximately [3.0, 4.5]
27 print(f"True gradient: [3.0, 4.5]")
```

# SPSA: Illustration and Comparison with Regression Gradient I



Each SPSA iteration uses one symmetric pair of perturbations  $\mathbf{x} \pm \delta \mathbf{z}$  (red and green dots). The estimated gradient direction (navy arrow) converges as more pairs are averaged.

**Reading the figure.** Each pair of colored dots represents one SPSA iteration: we step distance  $\delta$  in the random direction  $\mathbf{z}$  (green dot) and in the opposite direction (red dot). The difference in  $f$  values divided by  $2\delta$  gives a noisy slope, and multiplying by  $\mathbf{z}$  distributes that slope to all  $n$  gradient components. After averaging many pairs, the estimate converges to the true gradient.

# SPSA: Illustration and Comparison with Regression Gradient II

## SPSA vs. Regression Gradient: Summary Comparison

| Property                        | Regression Gradient           | SPSA                       |
|---------------------------------|-------------------------------|----------------------------|
| Evaluations per estimate        | $m + 1$                       | $2m$                       |
| Min. evaluations for 1 estimate | $n + 1$ (need $m \geq n$ )    | 2 (any $m \geq 1$ )        |
| Accuracy (fixed $m$ )           | Better (uses all directions)  | Noisier (random)           |
| Scales to large $n$ ?           | No ( $m$ must grow with $n$ ) | Yes (always 2 per iter)    |
| Typical use case                | Moderate $n$ , noisy $f$      | Very large $n$ , black-box |



# SPSA: Illustration and Comparison with Regression Gradient III

## Key Takeaway

SPSA is the preferred gradient estimator when  $n$  is very large and  $f$  is a black box. Each individual estimate is noisy, but the estimates are unbiased, so gradient-descent-style optimization algorithms converge on average. SPSA is widely used in reinforcement learning, neural architecture search, and the optimization of simulation-based systems.

# Chapter Summary: Core Concepts I

This chapter has built up a complete toolkit for computing or estimating the derivative of a function. We review the key ideas below.

## Mathematical Objects

- The **derivative**  $f'(x)$  of a scalar function  $f : \mathbb{R} \rightarrow \mathbb{R}$  measures the instantaneous rate of change of  $f$  at  $x$ : it is the slope of the tangent line.
- The **gradient**  $\nabla f(\mathbf{x})$  (nabla  $f$ ) of  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  is the vector of partial derivatives; it points in the direction of steepest ascent and has magnitude equal to the maximum rate of increase.
- The **Jacobian**  $\mathbf{J}$  of a vector-valued function  $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$  is the  $m \times n$  matrix of all partial derivatives; it generalizes the gradient to multi-output functions.
- The **Hessian**  $\mathbf{H}$  is the  $n \times n$  matrix of second-order partial derivatives; it captures the curvature of  $f$  and determines whether a critical point is a minimum, maximum, or saddle.
- The **directional derivative**  $\nabla_{\mathbf{s}} f(\mathbf{x})$  measures the rate of change of  $f$  in an arbitrary direction  $\mathbf{s}$  and equals  $\nabla f(\mathbf{x})^\top \mathbf{s}$ .

# Chapter Summary: Core Concepts II

## Differentiation Methods (Ordered by Accuracy and Cost)

- **Symbolic differentiation:** exact algebraic formula for  $f'$ ; requires a closed-form expression for  $f$ ; may suffer from expression swell.
- **Forward finite difference:** approximates  $f'$  with  $O(h)$  error using 2 evaluations; simple but limited by step-size trade-offs.
- **Central finite difference:**  $O(h^2)$  error, same 2 evaluations; more accurate because even-order Taylor terms cancel.
- **Complex-step method:** near-machine-precision accuracy, no subtractive cancellation; requires complex arithmetic support.
- **Forward-mode AD:** exact derivatives using dual numbers; one pass per input; efficient for few inputs.
- **Reverse-mode AD (backpropagation):** exact full gradient in one backward pass;  $O(\text{graph})$  memory; ideal for scalar loss with many inputs (deep learning).
- **Regression gradient:** estimates gradient from  $m + 1$  black-box evaluations via least squares; works for noisy or non-differentiable  $f$ ; cost grows with dimension.
- **SPSA:** unbiased gradient estimate from 2 evaluations regardless of dimension; noisier but

## Summary: Textbook Section 2.7 Bullet Points Explained I

The following points summarize the chapter's conclusions in the language of the textbook, with brief explanations.

- 1. Derivatives guide optimization.** Derivatives are useful in optimization because they provide information about how to change a given point to *improve* the objective function. Without this information, an optimizer must blindly probe the search space; with it, the optimizer can follow the gradient (or a Newton direction) to converge much faster.
- 2. Multivariate derivative objects.** For functions of multiple variables, the key derivative concepts are the **gradient** (first-order, direction of steepest change), the **Hessian** (second-order, curvature), and the **directional derivative** (rate of change along any chosen direction). Together these direct the search for an optimum.
- 3. Finite difference approximations.** One approach to numerical differentiation is to replace the theoretical limit in the derivative definition with a small but finite step  $h$ . The three basic variants — forward, central, and backward — differ in their accuracy: central difference achieves  $O(h^2)$  by exploiting symmetric cancellation of even-order error terms.

## Summary: Textbook Section 2.7 Bullet Points Explained II

**4. The complex-step advantage.** The **complex-step method** can eliminate subtractive cancellation error entirely. Because it extracts the imaginary part of a single complex evaluation rather than differencing two real evaluations, the step  $h$  can be made arbitrarily small (e.g.,  $10^{-20}$ ) without any loss of precision. The result is a near-machine-precision derivative estimate.

**5. Automatic differentiation.** **Automatic differentiation** computes derivatives of a program by systematically applying the chain rule to each elementary operation in the computational graph. Forward accumulation propagates tangents (one pass per input) using dual numbers; reverse accumulation (backpropagation) propagates adjoints (one pass for all inputs) by storing and replaying the computational tape. Both give exact floating-point derivatives.

**6. Regression gradient for black-box functions.** The **regression gradient** uses linear regression on a set of function evaluations at random perturbations to find the best-fit gradient vector. It works for any function that can be evaluated, even black-box, noisy, or non-differentiable ones, and requires  $m + 1$  evaluations.

**7. SPSA for high-dimensional black-box problems.** **SPSA** allows unbiased gradient estimation for very high-dimensional functions using only 2 function evaluations per sample, regardless of dimension. It is the method of choice when  $n$  is huge and  $f$  is a black box.

# Selected Exercises with Full Solutions I

**Exercise 2.1.** *How can you approximate the Hessian matrix from the gradient using forward differences?*

## Solution

The Hessian  $\mathbf{H}$  has entries  $H_{ij} = \partial^2 f / \partial x_i \partial x_j$ . We can estimate the  $j$ -th column of  $\mathbf{H}$  by applying forward difference to the gradient in the  $j$ -th direction:

$$\mathbf{H}_{:,j} \approx \frac{\nabla f(\mathbf{x} + h\mathbf{e}_j) - \nabla f(\mathbf{x})}{h}$$

where  $\mathbf{e}_j$  is the  $j$ -th standard basis vector. Repeating for each  $j = 1, \dots, n$  gives the full Hessian at cost  $n + 1$  gradient evaluations (each gradient costs  $n$  evaluations if computed by finite differences, for a total of  $O(n^2)$ ).

**Exercise 2.2.** *What is a drawback of central differences compared with forward differences?*

## Selected Exercises with Full Solutions II

### Solution

Central difference requires **two function evaluations per dimension** — one at  $x + h/2$  and one at  $x - h/2$  — whereas forward difference requires only one extra evaluation (at  $x + h$ , since  $f(x)$  is already known). So for computing the gradient in  $n$  dimensions, central difference costs  $2n$  evaluations vs.  $n + 1$  for forward difference. This doubling of cost may matter when  $f$  is expensive to evaluate.



## Selected Exercises with Full Solutions III

**Exercise 2.3.** Identify the dominant term in  $f'(x) = \frac{1}{x} + e^x - \frac{1}{x^2}$  for small  $x > 0$ .

### Solution

For small  $x$  (say  $x = 0.1$ ):

- $e^x \approx 1 + x \approx 1$  (small, bounded)
- $1/x = 10$  (large)
- $1/x^2 = 100$  (very large, and negative in the expression)

The term  $-1/x^2$  grows most rapidly as  $x \rightarrow 0^+$ , so  $-1/x^2$  dominates. The function approaches  $-\infty$  as  $x \rightarrow 0^+$ .

**Exercise 2.4.** If  $f(3 + ih) = 2 + 4ih$ , what is  $f'(3)$ ?

## Selected Exercises with Full Solutions IV

### Solution

By the complex-step formula:

$$f'(3) \approx \frac{\operatorname{Im}(f(3 + ih))}{h} = \frac{\operatorname{Im}(2 + 4ih)}{h} = \frac{4h}{h} = 4.$$

So  $f'(3) = 4$ . The real part gives  $f(3) = 2$ .

## Selected Exercises with Full Solutions V

**Exercise 2.6.** *Combine forward and backward differences to estimate  $f''(x)$ .*

### Solution

Observe that  $f''(x)$  is the derivative of  $f'(x)$ . We can apply finite difference to the derivative:

$$f''(x) \approx \frac{f'(x + h/2) - f'(x - h/2)}{h}$$

Now substitute the forward-difference approximation for each derivative:

$f'(x + h/2) \approx [f(x + h) - f(x)]/h$  and  $f'(x - h/2) \approx [f(x) - f(x - h)]/h$ . Substituting:

$$f''(x) \approx \frac{f(x + h) - 2f(x) + f(x - h)}{h^2}$$

This requires three function evaluations:  $f(x - h)$ ,  $f(x)$ , and  $f(x + h)$ . The truncation error is  $O(h^2)$ .

## Selected Exercises with Full Solutions VI

**Exercise 2.5.** Draw the computational graph for  $f(x, y) = \sin(x + y^2)$  and apply forward accumulation at  $(x, y) = (1, 1)$  to find  $\partial f / \partial y$ .

### Solution

The intermediate nodes are:  $c_1 = y^2 = 1$ ;  $c_2 = x = 1$ ;  $c_3 = c_2 + c_1 = 2$ ;  $c_4 = \sin(c_3) = \sin(2)$ . Seeding with  $\dot{y} = 1$ ,  $\dot{x} = 0$  and applying the chain rule at each node:

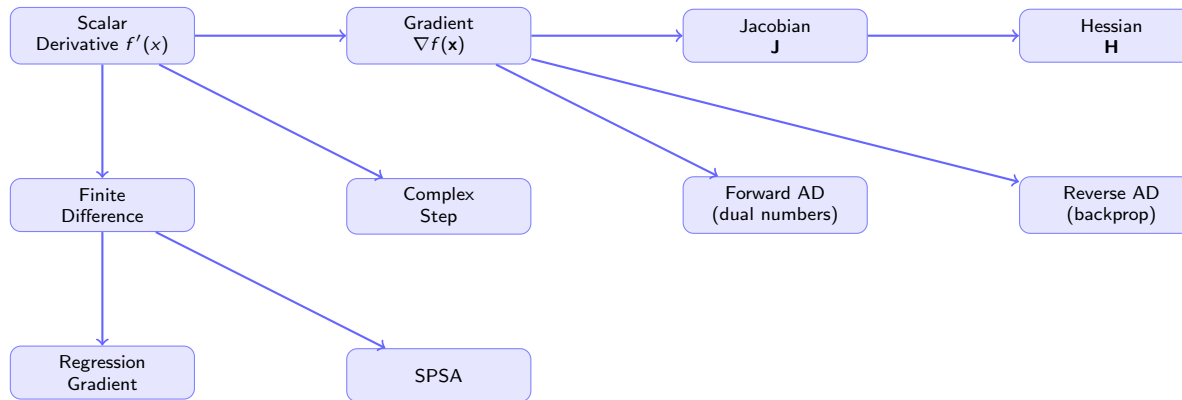
$$\dot{c}_1 = 2y \cdot \dot{y} = 2 \cdot 1 = 2; \quad \dot{c}_2 = \dot{x} = 0; \quad \dot{c}_3 = \dot{c}_2 + \dot{c}_1 = 0 + 2 = 2; \quad \dot{c}_4 = \cos(c_3) \cdot \dot{c}_3 = \cos(2) \cdot 2.$$

Therefore:

$$\frac{\partial f}{\partial y} = \dot{c}_4 = 2 \cos(2) \approx 2 \times (-0.4161) \approx -0.832.$$

The negative sign tells us that increasing  $y$  from  $y = 1$  while holding  $x = 1$  fixed decreases  $f$ .

## Chapter 2: Concept Map



To choose the right differentiation method for a given problem, consider the following decision criteria:

# Method Selection Guide II

## When to Use Which Method

| Scenario                                          | Recommended Method       | Reason                                  |
|---------------------------------------------------|--------------------------|-----------------------------------------|
| $f$ has a closed algebraic formula                | Symbolic differentiation | Exact; derivative is reusable           |
| Code-defined $f$ , few inputs ( $n$ small)        | Forward-mode AD          | One pass per input; exact               |
| Code-defined $f$ , scalar output, many inputs     | Reverse-mode AD          | One backward pass for full gradient     |
| Black-box $f$ , need high-accuracy gradient       | Complex-step             | Near-machine precision, no cancellation |
| Black-box $f$ , moderate $n$ ( $n \lesssim 100$ ) | Regression gradient      | Robust, overdetermined least squares    |
| Black-box $f$ , very large $n$                    | SPSA                     | Only 2 evaluations per estimate         |

# References

-  M. J. Kochenderfer and T. A. Wheeler, *Algorithms for Optimization*, 2nd ed. MIT Press, 2026.
-  J. R. R. A. Martins, P. Sturdza, and J. J. Alonso, “The Complex-Step Derivative Approximation,” *ACM Trans. Math. Softw.*, vol. 29, no. 3, pp. 245–262, 2003.
-  A. G. Baydin et al., “Automatic Differentiation in Machine Learning: a Survey,” *JMLR*, vol. 18, no. 153, 2018.
-  J. C. Spall, “Multivariate Stochastic Approximation Using a Simultaneous Perturbation Gradient Approximation,” *IEEE Trans. Autom. Control*, vol. 37, no. 3, pp. 332–341, 1992.